

SKILLS

Languages: Python, JavaScript/TypeScript, Java, C/C++, SQL, HTML/CSS**Frameworks:** PyTorch, Pandas, NumPy, Matplotlib, Scikit-learn, XGBoost, Flask, React, Node.js, LangGraph**Tools & Platforms:** AWS (S3, EC2, DynamoDB), Docker, CI/CD, Kubernetes, Linux, Git, VercelEDUCATION

University of Waterloo

Sep 2025 – Apr 2030

Bachelor of Mathematics

Relevant Coursework: Data Structures & Algorithms, Designing Functional Programs, Linear Algebra, Probability & Statistics

EXPERIENCE

AI Engineering Intern | *ProCogia*

May 2026 – Present

- Engineered a **fully open-source voice AI agent** POC from scratch in under a week — migrating off **Amazon Connect** to **LiveKit**, **Moonshine STT**, and a **locally hosted Qwen-32B LLM** on EC2 — eliminating **vendor lock-in** and replacing usage-based cloud costs with a fixed compute footprint.
- Reduced **latency to under 500ms** — a **3× improvement** over the initial prototype — by re-architecting the agent pipeline across **LangGraph** and **Twilio** with a modular, **distributed** design.
- Improved **RAG workflow** retrieval accuracy by designing and deploying **production-grade API integrations** and cloud infrastructure on **AWS (S3, EC2, DynamoDB)**, enabling scalable document ingestion for client workloads.
- Built an **automated smoke test suite** for a production **AWS** pipeline — validating **Lambda** health, **SQS** queue state, **DLQ** depth, and **DynamoDB** claim seeding in a single runner — replacing manual verification across **4 system boundaries**.

ML Engineer | *Wat Street (UWaterloo's Quantitative Finance Team)*

Jan 2025 – Present

- Reduced **model training iteration time by 40%** by engineering a scalable **data pipeline** (**Python, Pandas, NumPy**) that ingests live **S&P 500** market data and produces structured **tensor inputs** across **500 equities** per batch.
- Identified the best-performing volatility forecaster — cutting **prediction error by 18%** vs. the naive baseline — by benchmarking **3 ML models** (**HAR-RV, GARCH(1,1), LSTM**) and building a reusable **RMSE** evaluation harness.
- Improved **model reliability** by writing **unit tests** for pipeline stages, conducting **code reviews**, and debugging model discrepancies — catching **3 data leakage bugs** before evaluation.

Software Developer | *Waterloo Aerial Robotics Group (WARG)*

Oct 2025 – Apr 2026

- Increased ground control station coverage by **15%** by implementing a **coordinate mapping system** in **JavaScript** for **IMACS-3.0**, integrating it into the existing **backend architecture** and validating against live **flight telemetry**.
- Cut field debugging time by building a **real-time flight monitoring dashboard** in **Dart/Flutter**, displaying live telemetry and alerts across **Android and desktop** using **OOP** patterns for reusable, testable components.

PROJECTS

Flight Price Predictor | *Python, SQLite, XGBoost, Pandas, NumPy, Scikit-learn* | 

- Building an **end-to-end flight price prediction system** covering **200+ routes** across **55+ airports** — training an **XGBoost gradient-boosted model** on engineered features (route distance, lead time, day-of-week, airline type, transfer count) informed by **18 plots of documented EDA** across two data phases, targeting **sub-10% MAPE**.
- Built a **fault-tolerant ingestion pipeline** (**Python + SQLite + launchd**) pulling **~4,000 flight offers/day** over a projected **four months** — **retry-with-backoff** on **RequestException** reduced **final failure rate to 0%** across multiple slow-API days, with **automated cloud backup** and **per-run logging**.
- Maintained **dataset quality** by authoring automated **SQL deduplication** and **integrity-check scripts** running daily after ingestion — **9-column exact-key dedup guard**, **NULL/range audits** on modeling-critical fields, and **volume-drop detection** against a 3-day rolling baseline; **0 anomalies flagged** across **60K+ rows** to date.

AI Chess Bot | *Python, PyTorch, Flask, React/TypeScript, Docker* |  | 

- Deployed a **full-stack chess AI** as a single **Docker** image on **Render**, serving a **PyTorch** neural network (**768→256→128→1**) via **Flask REST API** and **React/TypeScript** frontend — **sub-100ms inference latency**, publicly accessible.
- Improved **model generalization** (**30% reduction in validation loss**) across **4 training iterations** — from a baseline evaluator to a tactical model — using **Stockfish**-labeled data, **Adam** optimizer, **dropout**, **batch normalization**, and hand-crafted signals: material balance, castling rights, and center control.